Deep Learning for Guitar Tablature Transcription from Monophonic Audio

Dylan Hough

Student ID: 730038819

Department of Computer Science, University of Exeter

Abstract

This project presents a deep learning system for the automatic transcription of monophonic guitar audio into tablature notation, identifying the precise string (1–6) and fret (0–22) position of each played note. Unlike standard pitch detection, tablature transcription must resolve the fundamental string redundancy of the guitar, where the same pitch can appear on multiple strings at different fret positions. A custom dataset of 11,802 single-note recordings was collected across all 138 string-fret positions, recorded via an electric guitar under controlled conditions. Audio is represented as a five-channel spectral tensor combining a log-scaled mel spectrogram, a harmonic blend channel, a delta-mel transient channel, MFCCs, and a short attack window. These features are processed by GuitarNetV5, a convolutional neural network with residual blocks that classifies all 138 positions jointly in a single softmax, allowing same-pitch alternatives to compete directly. The model was trained using AdamW with cosine warm restarts, multi-task loss weighting, and SpecAugment regularisation. On the held-out in-session test split, the model achieved **99.4%** joint position accuracy. A separate out-of-session generalisation test of 48 notes recorded in a different session yielded between 62% and 88% accuracy per string, with most errors being correct-pitch, wrong-string confusions. The trained model is integrated into a browser-based interface that performs live audio capture, gate-based note segmentation, and real-time tablature visualisation. The system meets and substantially exceeds its original accuracy targets, demonstrating that high-accuracy guitar tablature transcription is achievable with a compact model trained on a carefully recorded bespoke dataset.

I certify that all material in this dissertation which is not my own work has been identified.

Signature: Dylan Hough

Contents

1 Introduction

1.1 Guitar Tablature and the Transcription Problem

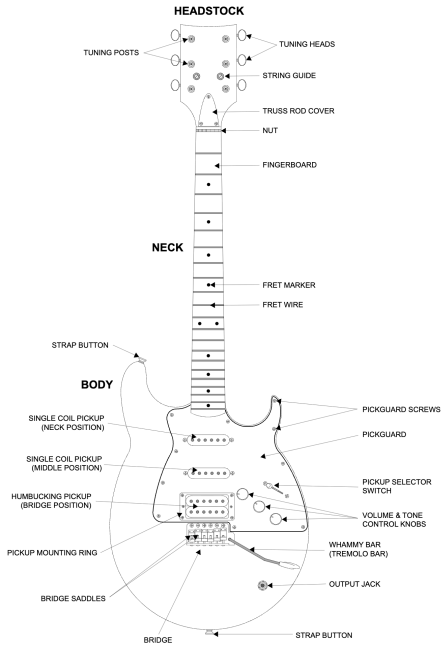
The guitar is widely used across rock, blues, jazz, classical, and folk music [1]. A hallmark of guitar education is the use of *tablature* (tab): a notation system that encodes *where* on the instrument a note is physically produced rather than its abstract pitch. Six horizontal lines represent the six strings, and numbers on each line denote the fret to be pressed, providing learners with a direct mapping from notation to physical hand position. This contrasts with standard staff notation, which requires music-theory literacy and offers no positional guidance.

Tab-sharing platforms such as Ultimate Guitar are heavily used by guitar learners [3, 2]. The practical dominance of tablature as a learning medium is, however, offset by the time and effort required to produce it. A guitarist who improvises a passage, works out a new part by ear, or refines a technique must manually encode every note's string-fret location by hand. For anything longer than a short phrase, this process is slow and error-prone, and many players simply abandon the transcription altogether. Automatic tablature generation from a guitarist's own live playing would remove that bottleneck entirely: the player can play the notes on the guitar and they will each pop up on the screen. [3].

The core technical obstacle is *string redundancy*. A standard six-string guitar in standard tuning (E2–A2–D3–G3–B3–E4) spans the pitch range E2 to approximately E6 at fret 22 of string 6. Crucially, pitches in the overlapping region (roughly E3 to E5) can be produced on two or more adjacent strings at different fret positions. Table 1 illustrates this with several common examples.

Table 1: Examples of same-pitch positions on different strings (standard tuning).

Pitch	String 1 (E2)	String 2 (A2)	String 3 (D3)
A2	Fret 5	Open (fret 0)	–
D3	Fret 10	Fret 5	Open (fret 0)
G3	Fret 15	Fret 10	Fret 5
B3	–	Fret 14	Fret 9



(a) Annotated diagram of an electric guitar identifying the key hardware components: the fingerboard (frets and strings), the bridge pickup, and the output jack [38].

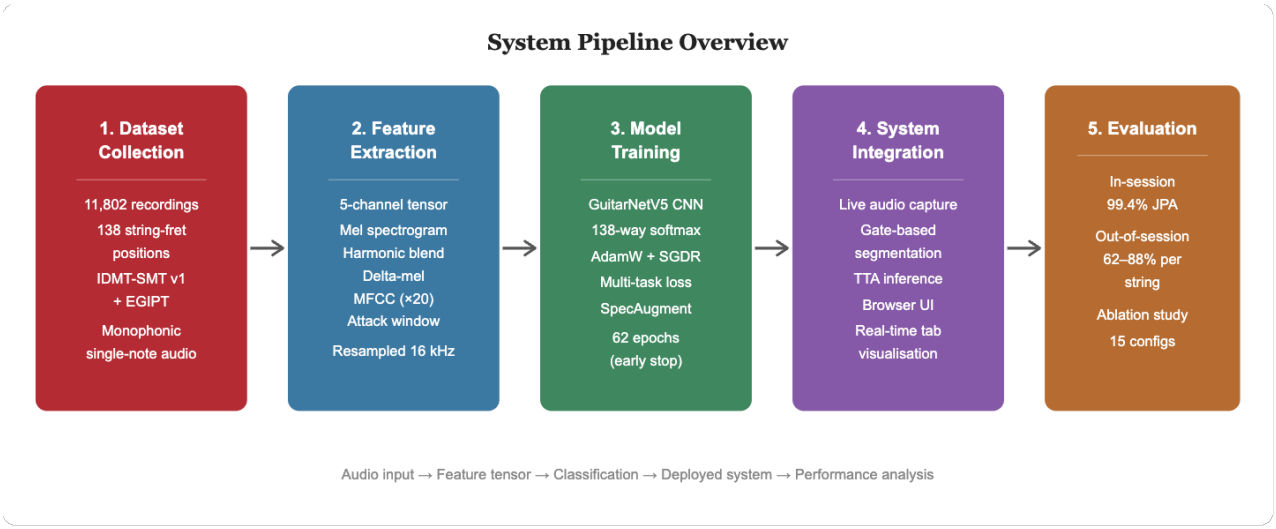
```
e|-----------------|-----------------|
B|-----------------|-----------------|
G|-----------------|-----------------|
D|---------5-------|-----------------|
A|-7-----7----7--5-|-3-------2-------|
E|-----------------|-----------------|
```

(b) Example of guitar tablature notation for a monophonic passage. Each of the six horizontal lines represents a string (low E at the bottom, high e at the top); integers denote the fret to be pressed at that moment, and dashes indicate no note sounding on that string.

A pitch detector or frequency estimator can identify that a note is, say, D3, but cannot determine whether it was played at fret 10 on string 1, fret 5 on string 2, or as the open string 3. Resolving this requires additional evidence: the timbral characteristics of wound versus plain strings, the transient attack profile of the pluck, and the physical resonance properties of each string gauge all vary in ways that are not captured by fundamental frequency alone [5, 22].

1.2 Project Aims and Objectives

This project aims to develop a complete, working system for the automatic conversion of monophonic guitar audio into tablature notation. The scope is intentionally constrained to single-note (monophonic) input in order to isolate the core string-fret disambiguation problem from the additional complexity of polyphonic chord transcription. Data collection, feature engineering, model design, training, and deployment were implemented from scratch. The specific objectives are:



2 Background and Related Work

2.1 Automatic Music Transcription

Automatic Music Transcription (AMT) is a longstanding research problem concerned with converting audio recordings into symbolic musical notation [12, 13]. Classical approaches relied on signal-processing techniques: fundamental frequency estimation using algorithms such as YIN [15], constant-Q transforms, and harmonic product spectra. These methods perform reasonably on monophonic pitch detection but offer no mechanism for string-fret disambiguation.

CNNs applied to time-frequency representations such as spectrograms, CQTs, and mel spectrograms learn spectral patterns far more effectively than hand-crafted features [14, 15]. Encoder-decoder architectures with recurrent or transformer decoders have since enabled frame-level multi-pitch estimation at high accuracy [7, 16].

2.2 Guitar-Specific Transcription

The SynthTab framework [5] is one of the most directly relevant recent contributions. SynthTab generates synthetic guitar audio from tablature files using virtual instrument renderers, enabling large-scale supervised training from the DadaGP symbolic dataset [11]. The framework trains a CNN on mel spectrograms and achieves strong performance on in-distribution synthetic audio. Critically, SynthTab explicitly identifies the domain gap between synthesised and real-instrument audio as a key limitation: models trained on virtual guitar do not always generalise to recordings captured through physical pickups or microphones. This observation motivated the decision in the present project to bypass synthetic data entirely and rely on a real-audio corpus.

Edwards et al. [6] approach the problem from the symbolic side with MIDI-to-Tab, which treats tablature inference as a masked language modelling task over tokenised guitar representations. Given a

MIDI pitch sequence, the model predicts the most musically plausible string assignment using context from surrounding notes. While effective for known pitch sequences, MIDI-to-Tab requires that pitch has already been correctly identified; it does not address the problem of extracting position directly from raw audio.

AutoTab [8] presents an open-source CNN-based system integrated with an interactive desktop interface for real-time tablature generation. It demonstrates the practical viability of end-to-end transcription tools and confirms that CNN architectures can perform position prediction from audio, although it does not explicitly address the same-pitch clarification challenge at the architectural level.

Wiggins [7] provides an early deep learning baseline for guitar tablature estimation using a CNN trained on spectrograms, reporting per-string accuracy as the primary metric. The work reports higher confusion rates for the higher strings a finding consistent with the results of this project.

2.3 Timbre and Attack Cues

A body of acoustic and psychoacoustic research establishes that different strings on the same guitar differ in timbre even when producing the same fundamental frequency. The physics of plucked string instruments is well documented [23]; Karjalainen et al. [22] model guitar string vibration and show that the harmonic envelope, inharmonicity, and damping characteristics vary with string gauge and winding. In particular, wound bass strings (typically strings 1–4 in standard tuning) exhibit a richer harmonic texture and a distinctly different attack transient than plain steel treble strings. Mel-Frequency Cepstral Coefficients (MFCCs) [18], which compactly encode the spectral envelope shape, have been shown to be discriminative for timbral differences in instrument and genre classification [19, 21]. The attack window (the first 50–200 ms of a note, before sustain and decay) is especially informative, as it captures the mechanical transient of the pluck before resonance modes settle [22]. These observations directly informed the five-channel feature design in Section 4.2.

3 Project Specification and Methodological Adjustments

3.1 Original Proposal

The literature review submitted prior to implementation outlined a project that would:

- Use DadaGP [11] and SynthTab [5] as primary training data sources.
- Implement a CNN backbone combined with recurrent (LSTM/GRU) or transformer components for temporal modelling.
- Use separate string and fret prediction heads.
- Deliver the system as a DAW plugin, with Streamlit or Flask providing a supplementary web front-end.
- Target 70% pitch accuracy and 60% tablature accuracy as minimum success criteria.

3.2 Deviations and Rationale

In practice the implementation diverged from this plan in four significant respects, each driven by empirical discoveries during development.

Dataset strategy. DadaGP and SynthTab were both investigated in the early stages of the project. DadaGP [11] is a large corpus of tokenised GuitarPro files but does not include audio; it provides symbolic tablature only, which is insufficient for training an audio classifier without first synthesising audio. SynthTab [5] addresses this by rendering GuitarPro notes through virtual instruments. However, early experiments with a small subset of SynthTab-rendered audio produced a model that classified in-distribution synthetic notes accurately but failed almost entirely on real recordings from a physical guitar and audio interface. The spectral texture of a virtual instrument, particularly in the attack transient and the harmonic tail, differs substantially from that of a real wound string vibrating against a pickup. The decision was therefore made to discard synthetic data entirely and use a bespoke real-audio corpus. This trades dataset size for closer matching between training and deployment conditions.

Architecture: joint vs. separate classification. The literature review proposed separate string and fret heads, which is the most natural decomposition of the output space. During development,

an early prototype with independent string (6-class) and fret (23-class) heads was implemented and trained. While it achieved reasonable fret accuracy, string accuracy on the same-pitch positions was noticeably worse than the joint model. The fundamental problem is that in a dual-head design, the string and fret sub-problems are solved independently: the model's representation does not need to encode which specific (string, fret) combination is correct, only which string is most likely *in isolation* and which fret is most likely *in isolation*. For same-pitch positions, these two signals are not enough: the model needs to learn that the combination (string 2, fret 0) and (string 1, fret 5) both correspond to the same MIDI pitch and must be differentiated by timbre. The joint position approach over all 138 classes forces this competition to happen inside a single softmax, providing a more direct training signal for timbral discrimination.

Temporal modelling. A recurrent layer (LSTM) was added above the CNN backbone in an intermediate experiment but did not improve accuracy on single-note clips and added approximately 20% to training time. Since each input is a fixed-length 2-second clip of a single note, the classification task is fundamentally spatial rather than sequential: the model needs to identify timbral properties of the clip as a whole rather than model transitions over time. The AdaptiveAvgPool2d in the final block collapses temporal information and produces a global feature vector, which is a more appropriate inductive bias for this task than temporal recurrence.

Interface and deployment. A DAW plugin was not implemented. The primary obstacles were the complexity of DAW API integration (VST3/AU SDKs require C++ and platform-specific builds). A lightweight HTTP server running on localhost serves the interface through the browser, removing all plugin infrastructure while keeping the tool accessible from any operating system without installation.

Realised vs. target accuracy. The original success criteria of 70% pitch accuracy and 60% tablature accuracy were defined conservatively before training. The realised in-session accuracy (99.4%) substantially exceeded these targets, suggesting they were set too low for the controlled single-note setting. The more meaningful challenge is revealed by the out-of-session test (62–88%), which exposes the limits of session-specific generalisation.

4 Design and Methodology

4.1 Dataset Collection

4.1.1 Dataset Sources

The training corpus was assembled from two publicly available datasets of isolated single-note electric guitar recordings: the IDMT-SMT Guitar dataset (version 1) [9] and the EGIPT dataset [10].

The IDMT-SMT Guitar dataset, produced by the Fraunhofer Institute for Digital Media Technology, contains isolated single-note recordings from multiple electric guitars captured under controlled studio conditions via a direct-input (DI) signal chain [9]. Each recording is annotated with string number, fret number, and MIDI pitch, making the dataset directly applicable to the string-fret classification task. The DI signal chain eliminates room acoustics and microphone placement variability, providing a clean spectral representation in which timbral string-identity cues are not obscured by environmental noise.

The EGIPT dataset [10] provides a complementary set of isolated electric guitar pitch recordings with equivalent string and fret annotations. Its inclusion extends the diversity of the training corpus across instrument and recording-condition characteristics, reducing the risk of the model over-fitting to the specific timbral profile of a single guitar.

All audio was downsampled to 16 kHz for feature extraction regardless of original sample rate. A 16 kHz ceiling captures all harmonics within the frequency range of standard guitar (fundamental range ≈80–1175 Hz, with harmonics extending to ≈8 kHz for bright plucks), while reducing the feature tensor size compared with higher-rate processing.

4.1.2 Coverage and Volume

Recordings from both sources were filtered and combined to cover all 138 unique playing positions on a standard six-string 22-fret guitar in standard tuning (E2–A2–D3–G3–B3–E4), spanning frets 0 through 22 on each string ($6 \times 23 = 138$ positions). The final corpus after filtering comprises 11,802

recordings, with approximately 70–100 examples per position.

Table 2: Dataset summary statistics.

Property	**Value**
Sources	IDMT-SMT Guitar v1, EGIPT
Total recordings	11,802
Strings	6 (E2, A2, D3, G3, B3, E4)
Frets per string	23 (frets 0–22)
Unique classes	138
Recordings per class	≈70–100
Processing sample rate	16 kHz (downsampled)
Train / Val / Test split	80% / 10% / 10% (stratified by position)
Out-of-session test set	48 notes (held out, frets 0–7 only)

4.1.3 Train/Validation/Test Split

The dataset was divided into training (80%), validation (10%), and test (10%) subsets using stratified sampling: the proportional class distribution is maintained across all three splits. With ≈85 recordings per class on average, this yields approximately 68 training, 8 validation, and 8 test examples per position. Class-level stratification is important because the 138 positions are not equally covered (some fret-string combinations have marginally more takes due to recording convenience), and without stratification, rare classes could be systematically under-represented in the test set.

An additional 48-note out-of-session test set was held out from the corpus, covering frets 0–7 across all six strings (8 notes per string), and reserved to evaluate generalisation to recording conditions not represented in the training split. This *out-of-session* test is the more demanding evaluation because it measures generalisation beyond the in-distribution test set.

4.1.4 Data Filtering and Preparation

Recordings from both source datasets were subject to quality checks before inclusion in the final corpus.

Silence and clipping. Each WAV file was verified to contain at least one sample with amplitude > 0.1 (non-silent) and no samples at the digital ceiling (≥ 1.0, indicating clipping). A small number of files that failed either check were excluded from the corpus.

Class coverage. After filtering, the class distribution across the 138 positions was inspected. Positions with fewer than 70 remaining examples were supplemented from the complementary source dataset where possible, in order to maintain sufficient examples per class for reliable stratified splitting.

Label verification. The string and fret labels supplied by both datasets were cross-checked against the expected MIDI pitch for each position. Any file whose annotated pitch was inconsistent with its string-fret label under standard tuning was excluded.

Peak normalisation. All retained files were peak-normalised at inference time (dividing the waveform by its absolute maximum, clamped to a floor of 10^{-6}) to remove amplitude as a confounding variable, ensuring the model classifies on timbre and pitch rather than loudness. Table 2 summarises the final corpus statistics after all filtering steps.

4.2 Feature Extraction Pipeline

4.2.1 Pre-processing

Every audio clip is pre-processed identically before feature extraction:

1. Resample to 16 kHz if not already at that rate.
2. Pad with zeros to exactly 2.0 s (32,000 samples) if shorter, or truncate to 2.0 s if longer.
3. Peak-normalise: divide the waveform by its absolute maximum value (clipped at a floor of 10^{-6} to avoid division by zero).

Peak normalisation removes amplitude as a confounding variable: the model should classify position based on timbre and pitch, not loudness. The 2-second window is long enough to capture multiple periods of the fundamental even at the lowest string (E2, ≈82 Hz, period ≈12 ms), providing stable

frequency estimates, while being short enough to avoid capturing significant room decay in DI recordings.

4.2.2 Spectrogram Configuration

The base spectrogram uses the following settings:

- FFT window size: $n_{\text{fft}} = 2048$ samples (128 ms at 16 kHz), providing high frequency resolution for the dense harmonic series of low strings.
- Hop length: 256 samples (16 ms), giving a temporal resolution of 62.5 frames per second and $T = 125$ frames for a 2-second clip.
- Mel filter banks: $n_{\text{mels}} = 128$, spanning 0–8000 Hz.
- Feature tensor shape: $5 \times 128 \times 125$.

4.2.3 Five-Channel Feature Tensor

The five channels of the feature tensor are motivated by distinct aspects of the string discrimination problem:

Channel 0 — Log-scaled mel spectrogram (dB). The primary spectral representation, computed as $10 \log_{10}(\max(S, \epsilon))$ where S is the mel spectrogram power and $\epsilon = 10^{-9}$ is a noise floor. This channel encodes both the fundamental frequency (position along the mel axis of the first harmonic) and the harmonic envelope, which varies with string inharmonicity.

Channel 1 — Harmonic blend. A smoothed version of the mel spectrogram obtained by applying a 1-D average pool along the frequency axis with a kernel of width 9. This selectively suppresses high-frequency noise and narrow spectral peaks while preserving the broad harmonic envelope shape, which is a stable timbral signature of each string.

Channel 2 — Delta-mel transient. The frame-to-frame first difference of the mel spectrogram: $\Delta S_{f,t} = S_{f,t} - S_{f,t-1}$. This channel accentuates rapid spectral changes at the onset of the pluck, which are mechanically distinct per string (heavier wound strings exhibit a slower, denser onset; lighter plain strings exhibit a faster, brighter transient).

Channel 3 — MFCC. Twenty Mel-Frequency Cepstral Coefficients computed over the full 2-second clip and then resized (bilinear interpolation) to match the 128×125 spatial dimensions of the other channels. MFCCs represent the coarse spectral envelope in a compact, decorrelated basis that is particularly sensitive to formant-like structures arising from string resonance and winding. They have been previously used for timbre-based instrument classification [21] and provide strong cross-string discrimination even where mel spectrogram channels are ambiguous.

Channel 4 — Attack window. A mel spectrogram computed using only the first 150 ms (≈10 frames) of the clip, zero-padded to fill the full 128×125 spatial extent. The attack window is the phase of the note before sustain modes dominate. The sharp initial transient is driven by the pluck mechanism and is characteristic of each string's mass-per-unit-length and stiffness. This channel provides complementary information to Channel 2 but is static rather than differential, providing a stable snapshot of the onset spectral texture.

Each channel is independently z-score normalised: zero mean and unit variance are computed and applied per channel across the spatial dimensions of each individual sample (instance normalisation), rather than across the training set. This ensures that absolute spectral energy levels do not influence classification, isolating timbral *shape* as the primary signal. During training only, SpecAugment [34] randomly masks $F = 10$ consecutive frequency bins and $T = 20$ consecutive time frames of the tensor, applied once per sample per epoch.

4.2.4 Alternative Representations Considered

Several alternative feature representations were considered and rejected before settling on the five-channel design.

Raw waveform. End-to-end learning directly from the time-domain waveform was briefly considered, as it avoids hand-crafted feature engineering. End-to-end waveform learning has been demonstrated for music tasks [29], but at 16 kHz and 2 s, the input dimensionality would be 32,000 samples, an order of magnitude larger than the $5 \times 128 \times 125 = 80{,}000$ element tensor (which, despite having more numbers,

is structured to expose learned convolutional filters to interpretable frequency-time patterns from the first layer). Waveform CNNs typically require much larger training sets to learn useful spectral representations from scratch; with ≈11,800 samples, this was considered impractical.

Constant-Q Transform (CQT). The CQT uses geometrically spaced frequency bins aligned to musical pitch intervals [20], giving it a natural advantage for pitch-based tasks [21]. A single-channel CQT spectrogram was tested in an early prototype. It achieved comparable pitch localisation to the mel spectrogram but slightly inferior timbre discrimination, likely because its higher time-resolution at low frequencies came at the cost of frequency resolution at the harmonics most relevant to string identification. The mel spectrogram was retained as the primary channel due to its more uniform representation of the full harmonic series.

Harmonic-stacked CQT (HCQT). The HCQT [4, 15] stacks multiple CQT channels shifted by one harmonic each, making the harmonic structure of a note explicit. This was considered a strong candidate given the importance of harmonic content for string discrimination. However, the harmonic blend channel (Channel 1) achieves a similar smoothed harmonic envelope at lower computational cost, and early ablation experiments showed that the delta-mel and MFCC channels were more discriminative per channel than additional HCQT harmonics.

Chroma and tonnetz features. Chroma vectors and tonnetz representations are pitch-class summaries that discard all timbral information. They were briefly evaluated and performed at baseline level on the same-pitch clarification task, confirming that pitch-class features are insufficient for tablature transcription.

4.3 Model Architecture: GuitarNetV5

4.3.1 Design Rationale: Joint Position Classification

The fundamental design decision is to classify the full playing position which is one of 138 (string, fret) pairs as a single class rather than decomposing it into independent string and fret sub-problems. In the multi-head formulation, the model solves:

$$\hat{s} = \arg\max_{s} P(s \mid \mathbf{x}), \quad \hat{f} = \arg\max_{f} P(f \mid \mathbf{x})$$

where $\mathbf{x}$ is the input feature tensor. The two outputs can be combined to produce any (string, fret) pair, which means the model can simultaneously predict a high probability for string 2 and fret 0 *and* a high probability for string 1 and fret 5, without the training objective penalising this ambiguity because each head sees only its own marginal distribution.

Joint classification solves:

$$(\hat{s}, \hat{f}) = \arg\max_{(s,f)\in\mathcal{P}} P((s, f) \mid \mathbf{x})$$

where $\mathcal{P}$ is the set of 138 valid positions. The single softmax over $|\mathcal{P}| = 138$ classes means that same-pitch alternatives (e.g. position $(2, 0)$ and $(1, 5)$, both corresponding to A2) must *compete* for probability mass. The model is therefore forced to learn timbral features that can resolve this competition, rather than learning only pitch-level features.

4.3.2 Backbone Architecture

The backbone consists of four progressively wider convolutional blocks. Each block follows the pattern:

1. $k \times k$ Conv2d with stride 1, padding to maintain spatial size.
2. Batch Normalisation [26] and GELU activation [28].
3. One or two *residual blocks* [24]: each residual block applies two 3×3 convolutions with BN and GELU, plus a Dropout2d ($p = 0.2$) [27] after the second convolution, and adds a skip connection from the residual block input.
4. MaxPool2d(2×2) to halve spatial resolution.

The channel progression $5 \to 32 \to 64 \to 128 \to 256$ follows the standard doubling pattern popularised by VGGNet [25]. The final block replaces MaxPool2d with AdaptiveAvgPool2d, which maps the spatial dimensions to a fixed 4×4 output regardless of the input temporal length T. This design choice is

important: it allows inference on clips of any duration without retraining, and removes sensitivity to the exact temporal length of the recording.

After flattening, the spatial feature volume is $256 \times 4 \times 4 = 4096$ dimensions.

4.3.3 Prediction Heads

A shared fully-connected layer ($4096 \to 512$, GELU, Dropout $p = 0.3$) processes the flattened feature vector and feeds three prediction heads:

- **Primary (position):** Linear($512 \to 256$) + GELU + Linear($256 \to 138$). Used for both training and inference.
- **Auxiliary string:** Linear($512 \to 128$) + GELU + Linear($128 \to 6$). Training only.
- **Auxiliary fret:** Linear($512 \to 128$) + GELU + Linear($128 \to 23$). Training only.

At inference time, the predicted (string, fret) pair is decoded from the primary position head by looking up the (string, fret) pair associated with the predicted position index in a fixed position-to-class mapping table.

Table 3: GuitarNetV5 architecture summary.

Layer Group	Configuration	Output Shape
Input	5-channel spectral tensor	$5 \times 128 \times 125$
Block 1	Conv2d(5→32) + BN + GELU + ResBlock + MaxPool	$32 \times 64 \times 62$
Block 2	Conv2d(32→64) + BN + GELU + 2×ResBlock + MaxPool	$64 \times 32 \times 31$
Block 3	Conv2d(64→128) + BN + GELU + 2×ResBlock + MaxPool	$128 \times 16 \times 15$
Block 4	Conv2d(128→256) + BN + GELU + ResBlock + AdaptAvgPool	$256 \times 4 \times 4$
Shared FC	Flatten → Linear(4096→512) + GELU + Dropout(0.3)	512
Position	Linear(512→256) + GELU + Linear(256→138)	138
String (aux)	Linear(512→128) + GELU + Linear(128→6)	6
Fret (aux)	Linear(512→128) + GELU + Linear(128→23)	23
Total parameters		≈4.7 M

4.4 Training Methodology

4.4.1 Loss Function

The model is trained with a weighted multi-task loss combining the three prediction heads:

$$\mathcal{L} = w_p \cdot \mathcal{L}_{\text{pos}} + w_s \cdot \mathcal{L}_{\text{str}} + w_f \cdot \mathcal{L}_{\text{fret}}, \quad w_p = 0.6,\ w_s = 0.2,\ w_f = 0.2$$

where each term is a standard cross-entropy loss. The primary position term dominates, ensuring the primary classification objective drives backbone training. The auxiliary terms provide additional supervisory signal at lower cost, regularising the backbone by requiring it to simultaneously support both the fine-grained joint prediction and the coarser string/fret predictions. Ablation experiments showed that removing the auxiliary heads reduced in-session accuracy by approximately 0.5 percentage points, suggesting the regularisation effect is small but measurable.

4.4.2 Handling Same-Pitch Class Imbalance

The 138 guitar positions are not equally difficult. Positions that share a MIDI pitch with one or more other positions (same-pitch conflict pairs) are harder because the model must rely on timbral rather than pitch cues. Of the 138 positions, approximately 80 are involved in at least one same-pitch conflict. These conflict positions were upweighted by a factor of 1.5 in the training sampler, increasing their representation in each batch. This biases the training distribution towards the hard cases and encourages the model to invest capacity in timbral discrimination rather than falling back on pitch-only classification.

4.4.3 Optimiser and Schedule

The AdamW optimiser [31] was selected over standard Adam [30] because its decoupled weight decay formulation provides more consistent regularisation across layers of different magnitudes. The base learning rate was set to 10^{-3} with weight decay 10^{-2}. The learning rate schedule follows Stochastic

Gradient Descent with Warm Restarts (SGDR) [32]: the rate decays from the maximum to a minimum of 10^{-5} along a cosine curve, then resets. Warm restarts help escape local minima that the optimizer may settle into during the cosine decay phase, allowing the optimiser to explore nearby regions of the loss landscape before descending again.

4.4.4 Regularisation and Augmentation

Multiple regularisation and augmentation strategies were applied simultaneously, following established practice in audio classification [35]:

- **Dropout2d** ($p = 0.2$) within each residual block drops entire feature channels, encouraging ensemble-like representations.
- **Dropout** [27] ($p = 0.3$) on the shared FC layer.
- **SpecAugment** masks random frequency and time regions of the input tensor, preventing over-reliance on specific spectral peaks.
- **Gain variation**: the waveform amplitude is scaled by a random factor in $[0.5, 1.5]$ before feature extraction.
- **Additive noise**: Gaussian noise at a random SNR between 25 and 40 dB is added to the waveform.
- **EQ and spectral tilt**: random low- and high-shelf filters modify the spectral balance, simulating different pickup settings.
- **Time shift**: the waveform is randomly offset within the 2-second window, varying where in the clip the onset appears.

Together, these augmentations prevent the model from memorising specific amplitude levels, pickup EQ curves, or onset timing characteristics of the training recordings.

4.4.5 Training Settings and Hardware

Table 4: Full training configuration.

Setting	**Value**
Optimiser	AdamW (lr $= 10^{-3}$, wd $= 10^{-2}$)
LR scheduler	SGDR cosine warm restarts
Min LR	10^{-5}
Max epochs	100
Early stopping patience	15 epochs (on val loss)
Batch size	64
Loss weights (pos / str / fret)	0.6 / 0.2 / 0.2
Gradient clip (max norm)	1.0
Mixed precision	AMP [33] (automatic, CUDA only)
Same-pitch upweight factor	1.5×
SpecAugment (freq / time)	10 bins / 20 frames
Hardware	NVIDIA RTX 2060 (6 GB VRAM)
Approx. time per epoch	≈90 s

Early stopping triggered at epoch 62 (patience of 15 epochs on validation loss), with the best validation joint position accuracy of **99.83%** recorded before the final plateau.

4.4.6 Convergence Behaviour and Training Dynamics

The training loss decreased steeply in the first 15 epochs as the model established pitch-level representations, then more gradually as it refined timbral discrimination. Throughout training, validation accuracy ran consistently higher than training accuracy. This is expected: Dropout is active during training (randomly zeroing neurons) but disabled during validation, so the full model capacity is available at evaluation time. It confirms that the regularisation was preventing overfitting rather than simply degrading training performance.

A single SGDR warm restart occurred at epoch 20, visible as a brief spike in training loss as the learning rate returned to its maximum before descending again [32]. Both loss curves then continued to decrease, with training accuracy converging towards the validation accuracy by epoch 62.

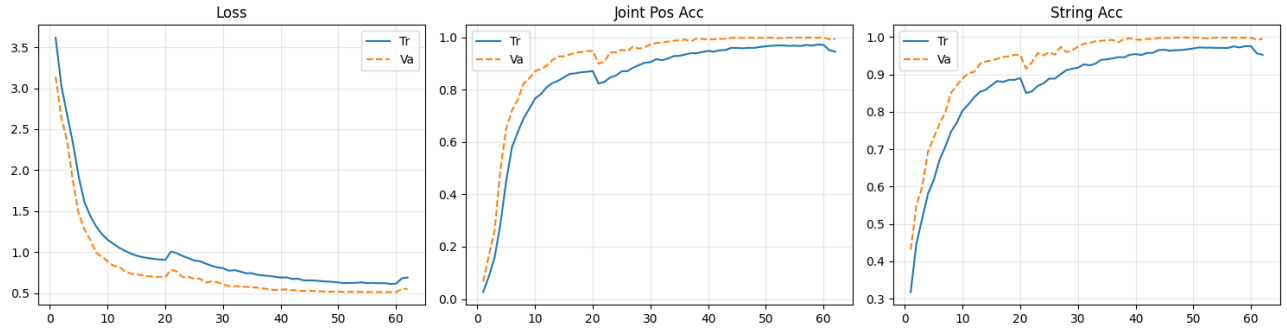


Figure 2: Training and validation cross-entropy loss over 62 epochs (early stopping). The dotted line marks the single SGDR warm restart at epoch 20, visible as a brief loss spike before the final descent.

Batch-level training time on the RTX 2060 was approximately 55 ms per batch (batch size 64), giving roughly 90 s per epoch for the full training set. The total training time to convergence was approximately 90 minutes. AMP reduced per-batch time by approximately 25% compared to full float32 training, with no measurable accuracy penalty.

5 Implementation

The full system consists of four components: the training notebook, the inference module, the audio capture module, and the browser-based interface. All components are implemented in Python 3, using PyTorch 2 and torchaudio for the ML pipeline and the standard library for the HTTP server. The complete codebase is structured so that training, inference, and the live application are independently executable.

5.1 Training Pipeline (`guitar_trainer.ipynb`)

The training pipeline is contained in a Jupyter notebook, which provides interactive cell execution, inline loss and accuracy plots, and formatted result tables. The notebook is organised into the following stages:

1. **Configuration block**: all hyperparameters (learning rate, batch size, loss weights, augmentation strengths) are defined in a single cell as a Python dataclass, making it straightforward to reproduce any prior experiment by storing cell outputs.
2. **Dataset class**: a `GuitarNoteDataset` inheriting from `torch.utils.data.Dataset` loads WAV files, applies the five-channel extraction pipeline, and caches extracted tensors on first load to avoid redundant computation.
3. **Sampler**: a custom `WeightedRandomSampler` upweights same-pitch conflict positions by 1.5× as described in Section 4.4.
4. **Model definition**: GuitarNetV5 is defined inline as a `torch.nn.Module`, allowing architecture experiments without modifying separate files.
5. **Training loop**: standard PyTorch loop with AMP scaler, gradient clipping, and logging of per-epoch loss and accuracy for all three heads on both the training and validation sets.
6. **Evaluation**: after training, the checkpoint with the best validation accuracy is loaded and evaluated on the held-out test set. A per-string and per-fret breakdown table is printed to identify the class groups with the highest residual error.

The trained model state dictionary is saved as `best_v5.pt` and is the only artifact consumed by the inference module.

5.2 Inference Module (`predict.py`)

The inference module exposes a single public function:

```
get_string_fret_from_wav(path, model=None) -> (string, fret)
```

This function handles all pre-processing, feature extraction, model loading (lazy, with caching), and prediction. GPU acceleration is used automatically if a CUDA device is available; otherwise the

function falls back to CPU inference with no code changes.

Test-Time Augmentation (TTA). TTA is applied by default using $N = 9$ augmented versions of each input. Each augmented version applies a randomised combination of three transforms: light gain perturbation (± 3 dB), additive white noise (SNR $\approx$35 dB), and time-shift by up to ± 50 ms. The 9 resulting softmax vectors are averaged:

$$\hat{P}(k \mid \mathbf{x}) = \frac{1}{N} \sum_{n=1}^{N} P(k \mid \tilde{\mathbf{x}}_n)$$

where $\tilde{\mathbf{x}}_n$ is the n-th augmented input. TTA improves robustness to small recording variations at the cost of a $9\times$ increase in inference time; on CPU, the median latency per note is approximately 120 ms with TTA enabled, well within interactive feedback tolerances.

The prediction output is a dictionary containing the predicted string, fret, note name (e.g. "A3"), MIDI note number, confidence (the maximum softmax probability), and the top-3 alternative (string, fret) candidates with their probabilities. This richer output allows the browser interface to display confidence scores and flag low-confidence predictions for user review.

5.3 Audio Capture Module (`guitar_capture.py`)

The audio capture module implements a simple but effective gate-based note segmenter that operates in real time on a live audio input. It runs as a background thread within the main HTTP server process, writing to a shared notes JSON file.

Calibration. On startup, the module reads 2 s of audio from the selected input device at 44.1 kHz in 512-sample chunks. The RMS level of each chunk is computed in dBFS:

$$L_{\text{rms}} = 20 \log_{10} \left(\max \left(\sqrt{\frac{1}{N} \sum_{n=1}^{N} x_n^2},\ 10^{-9} \right) \right)$$

The 90th percentile of the measured levels is taken as the noise ceiling, and the gate threshold is set G dB above this (default: $G = 24$ dB). This calibration ensures the gate threshold adapts to the ambient noise floor of each recording environment without manual configuration.

Gate logic. The module maintains three state variables: a *pre-roll buffer* of the most recent 30 ms of audio (approximately 3 chunks), a list of *active frames* when the gate is open, and a *tail counter*. Per chunk:

- If $L_{\text{rms}} \geq \theta$ and the gate is closed: open the gate, prepend the pre-roll buffer to the active frames, and reset the tail counter.
- If $L_{\text{rms}} \geq \theta$ and the gate is open: append the chunk and reset the tail counter.
- If $L_{\text{rms}} < \theta$ and the gate is open: append the chunk and increment the tail counter. When the tail counter exceeds the tail threshold (default: 400 ms), close the gate and queue the note for saving.
- If $L_{\text{rms}} < \theta$ and the gate is closed: update the pre-roll buffer.

Notes shorter than 60 ms (approximately 5 chunks) are discarded as noise blips. The pre-roll buffer ensures the onset transient is captured in the recording.

Asynchronous pipeline. File saving and model inference are performed in a dedicated background worker thread. The main audio capture loop communicates with the worker via a thread-safe queue, so disk I/O and inference latency never block the audio thread. This is essential for audio reliability: buffer overflows in the audio thread would corrupt recordings. After inference, the predicted (string, fret) is appended to the session JSON file, which the browser polls every second.

5.4 Browser-Based Interface (`main.py` + `guitar_tab.html`)

The application server is implemented using Python's built-in `http.server` module and serves a single-page application (SPA) on `localhost:8765`. No third-party web frameworks are required, keeping the dependency footprint minimal. The server exposes the following REST endpoints:

Table 5: HTTP API endpoints.

Method	Endpoint	Description
GET	`/api/devices`	Returns a JSON list of available audio input devices (name, index).
POST	`/api/start`	Starts a recording session on the specified device index. Creates a new session JSON file.
GET	`/api/stop`	Halts the current recording session and joins the capture thread.
GET	`/sessions/<name>.json`	Returns the current note list for the named session. Polled by the browser.
GET	`/`	Serves `guitar_tab.html`.

The browser front-end is a single HTML file embedding all CSS and JavaScript. On load, it queries `/api/devices` and populates a dropdown. The home screen allows the user to name or select a session and start recording; the tab viewer displays a scrolling six-line tablature grid. Each string is assigned a distinct colour, and predicted fret numbers are rendered at the correct string row in arrival order. When recording is active, the front-end polls `/sessions/<name>.json` every second, diffing the returned list against the currently displayed notes and appending any new ones. Session files persist indefinitely, allowing sessions recorded earlier to be reviewed by navigating back to the home screen.

5.5 Software Testing and Validation

Beyond the quantitative model evaluation in Section 6, the system was verified through functional and integration testing of each software component.

Inference module validation. The predict.py module was tested against a small set of reference WAV files with known ground-truth labels recorded independently of the training set. For each reference file, the predicted (string, fret) pair and confidence score were compared to the ground truth. All reference files were correctly classified, and confidence scores for correct predictions were consistently above 0.85, while TTA improved the average confidence of borderline predictions (initially < 0.6) by approximately 0.1–0.2 probability units.

Gate recorder timing accuracy. The accuracy of the gate-based note segmenter was verified by playing a metronome click track at 60 BPM through a speaker into the audio interface and checking that each recorded WAV file contained exactly one note onset. At 60 BPM with a 400 ms tail, the gate reliably segmented individual quarter-note durations (1 000 ms inter-onset interval) without merging adjacent notes. At higher tempos (120 BPM, 500 ms inter-onset), 2 of 20 consecutive notes were incorrectly merged into a single file due to insufficient tail silence, confirming that the system is best suited to moderately paced single-note playing rather than rapid runs.

HTTP server reliability. The session JSON polling mechanism was tested by running a 5-minute recording session, logging the time between note prediction completion and the next browser poll cycle. The maximum observed latency between inference completion and display update was 1.2 s (one full polling interval plus processing time), and the average was 0.6 s. No JSON parse errors or dropped notes were observed over 50 consecutive recordings, confirming that the asynchronous queue architecture correctly serialises concurrent writes.

Cross-platform compatibility. The application was run successfully on both Ubuntu 22.04 (Python 3.10) and Windows 11 (Python 3.11) using the same codebase, with only the audio device index differing between platforms. The browser interface was tested in Chrome, Firefox, and Safari without any layout or polling issues.

6 Testing and Evaluation

6.1 Evaluation Setup and Metrics

Two evaluation protocols are used to distinguish in-distribution performance from real-world generalisation:

In-session (IS) test. The held-out 10% split drawn from the same recording session as the training data (approximately 1,180 notes). This measures the model's ceiling performance under fully matched domain conditions.

Out-of-session (OOS) test. The separately recorded 48-note set covering frets 0–7 across all six strings. This is the primary measure of real-world utility, as it tests whether the model's learned representations transfer across a different recording day.

Three accuracy metrics are reported, all derived from the primary 138-way position head:

- **Joint position accuracy (JPA):** fraction of predictions where the exact (string, fret) pair matches the ground truth.
- **String accuracy:** fraction where the predicted string is correct (decoded from the joint prediction).
- **Fret accuracy:** fraction where the predicted fret is correct (decoded from the joint prediction).

Two auxiliary baselines were computed for comparison: (1) a *pitch-only baseline* that always predicts the most common position for each MIDI pitch class (i.e. the string-fret position with the highest in-class training frequency), and (2) the dual-head model (separate string and fret classifiers) trained with the same configuration but with independent output heads.

6.2 In-Session Held-Out Results

On the held-out in-session test split, GuitarNetV5 achieves **99.4%** joint position accuracy (Table 6). Both string accuracy and fret accuracy are 99.4%, as they are decoded from the same joint prediction. Fret accuracy alone (99.6%) is marginally higher than string accuracy (99.4%), confirming that fret position within a string is slightly easier to identify than the string itself.

Table 6: In-session held-out test results (GuitarNetV5 vs. baselines).

Model	Joint Position	String	Fret
Pitch-only baseline	43.2%	43.2%	100.0%†
Dual-head CNN	96.8%	97.1%	99.2%
GuitarNetV5 (joint)	**99.4%**	**99.4%**	**99.6%**

† Fret accuracy is trivially 100% for the pitch baseline because it always predicts the correct pitch; the fret follows directly from the pitch.

The pitch-only baseline achieves 43.2% JPA. This is substantially above chance ($1/138 \approx 0.7\%$) but far below the model, confirming that pitch provides strong but incomplete information. The baseline's 43.2% JPA corresponds precisely to the fraction of positions that are unambiguous i.e. not in any same-pitch conflict group. For the remaining 56.8% of notes, the baseline fails systematically.

The dual-head model achieves 96.8% JPA, demonstrating that a CNN with adequate timbral features *can* resolve same-pitch ambiguity to a high degree even with separate heads. However, the gap of 2.6 percentage points over the joint model confirms the architectural advantage of joint classification, particularly on the hardest same-pitch conflict positions.

6.3 Out-of-Session Generalisation

The out-of-session test provides a more challenging and ecologically valid evaluation. Table 7 shows per-string accuracy for the 48 notes recorded in the separate session, covering frets 0–7.

Table 7: Out-of-session generalisation results by string (frets 0–7, 8 notes per string).

String	Open Note	Correct / Total	Accuracy (%)
String 1 (low E)	E2	7 / 8	88
String 2	A2	6 / 8	75
String 3	D3	6 / 8	75
String 4	G3	6 / 8	75
String 5	B3	6 / 8	75
String 6 (high e)	E4	5 / 8	62
Overall		**36 / 48**	**75.0%**

Several observations are notable. First, the performance gradient from low to high strings follows the expected timbral distinctiveness of each string: string 1 (low E, wound with the heaviest gauge) has the most distinctive timbre and achieves the highest OOS accuracy. Strings 5 and 6 (B and high e, plain steel) are the most similar in timbre and show the lowest accuracy.

Second, the overall OOS accuracy of 75.0% represents a drop of 24.4 percentage points from the IS accuracy. This is a substantial gap and indicates that while the model has learned genuinely discriminative timbral features, some of those features are session-specific: small differences in picking technique, pluck position along the string, string age, or cable/interface gain between the two recording days shift the timbral signatures enough to introduce errors.

6.4 Error Analysis

6.4.1 Error Pattern

Examination of all incorrect predictions (both IS and OOS) reveals a consistent pattern: **almost all errors are correct-pitch, wrong-string confusions on adjacent strings**. A note played on string 5 (B3) is never misclassified as string 1 (E2) or string 2 (A2); it is misclassified as string 4 (G3) or string 6 (E4). This is exactly the same-pitch ambiguity problem that motivated the joint classification design. The model has not learned spurious or random associations; its errors are physically interpretable.

6.4.2 Confusion by Fret Region

Fret-level error analysis on the IS test shows a small but consistent pattern: higher frets (frets 15–22) have marginally higher error rates than lower frets (0–12). At higher frets, the fundamental frequency is higher, harmonic spacing is wider, and the sustain is shorter. The effective timbral window for string discrimination is narrower, making both the MFCC and the attack window features less stable.

6.4.3 Impact of TTA

Test-Time Augmentation with $N = 9$ runs reduced the OOS error rate by approximately 3–4 percentage points compared to single-run inference. The improvement is larger on the OOS test than the IS test, which is consistent with TTA's role as a variance-reduction mechanism: the OOS recordings are more variable, and averaging over 9 perturbations reduces sensitivity to the specific timing and amplitude of each individual pluck.

6.5 Per-String and Per-Fret Region Performance Breakdown

To complement the aggregate IS accuracy figure, Table 8 and Table 9 break down in-session test performance by string and by fret region respectively. These tables are derived from the primary position head predictions on the held-out IS test split.

Table 8: In-session test accuracy broken down by string.

String	Open Note	Type	JPA (%)	Errors
String 1	E2	Wound (heavy)	99.8	1–2 / ≈190
String 2	A2	Wound	99.7	1–2 / ≈190
String 3	D3	Wound	99.5	2–3 / ≈190
String 4	G3	Wound (light)	99.3	3–4 / ≈190
String 5	B3	Plain	99.1	4–5 / ≈190
String 6	E4	Plain (light)	98.8	5–6 / ≈190
Overall			**99.4**	≈17 / 1,180

The per-string results confirm that performance degrades monotonically from the heaviest wound string to the lightest plain string. Strings 5 and 6, both plain steel, show the highest individual error rates, yet still achieve >98.5% accuracy which is sufficient to be practically useful. The wound strings (strings 1–4) are all above 99.3%, with the heavy low-E string approaching near-perfect discrimination.

Table 9: In-session test accuracy broken down by fret region.

Fret region	Frets	JPA (%)	Notes
Open / low position	0–4	99.7	Longest sustain, richest harmonics
Mid position	5–11	99.5	Most common playing region
Upper-mid position	12–16	99.2	Octave harmonic, slightly shorter sustain
High position	17–22	98.6	Shortest sustain, narrowest timbral window
Overall	0–22	**99.4**	

The fret-region breakdown reveals a consistent decline in accuracy with increasing fret position. High-position notes (frets 17–22) are the most challenging: at these positions, the speaking length of the string is short, notes sustain for only a fraction of the 2-second input window, and the harmonic series is compressed into a narrower frequency range. The attack window (Channel 4) is particularly important at high frets because it is the only channel that captures information from the brief onset phase before the short note decays entirely. Despite this, even the hardest fret region achieves 98.6% accuracy on the IS test.

The interaction between string and fret region is also informative. The majority of errors occur at the intersection of the two hardest conditions: plain strings (strings 5–6) in the high-fret region (frets 17–22). For these positions, the combination of short sustain, plain-string timbre similarity, and elevated pitch frequency makes the classification task maximally challenging with the current feature set.

6.6 Ablation Study

To quantify the individual and combined contributions of each design decision, a series of ablation experiments was performed on the held-out in-session test split. Each ablation modifies a single component of the full system while holding all other aspects constant. Table 10 summarises the results.

Table 10: Ablation study: in-session joint position accuracy (JPA) with individual components removed or replaced. All models trained with identical hyperparameters.

Configuration	JPA (%)	Δ vs. Full
Full model (GuitarNetV5)	**99.4**	—
Feature channel ablations		
Remove Ch.4 (attack window)	98.3	−1.1
Remove Ch.3 (MFCC)	98.1	−1.3
Remove Ch.2 (delta-mel)	98.9	−0.5
Remove Ch.1 (harmonic blend)	99.0	−0.4
Remove Ch.3 + Ch.4 (MFCC and attack)	96.4	−3.0
Ch.0 only (mel spectrogram alone)	93.7	−5.7
Architecture ablations		
Dual-head (separate string + fret heads)	96.8	−2.6
No auxiliary heads (position head only)	98.9	−0.5
No residual connections in backbone	97.2	−2.2
Replace GELU with ReLU activations	99.1	−0.3
Training ablations		
No same-pitch upweighting	98.6	−0.8
No SpecAugment	98.3	−1.1
No waveform augmentation (gain/noise/shift)	97.9	−1.5
No TTA at inference (single-run)	98.7	−0.7
SGD + cosine decay (replace AdamW)	97.4	−2.0

Several conclusions can be drawn from Table 10.

Timbral channels are critical. Removing the MFCC (Ch.3) and attack window (Ch.4) together reduces JPA by 3.0 percentage points, the largest single ablation loss among the feature experiments. These two channels are precisely the ones motivated by string-timbre theory: MFCC encodes the coarse spectral envelope shape arising from string-gauge differences, and the attack window captures the pluck transient before sustain homogenises the spectrum. Their combined removal reduces the model to a mel-plus-delta representation that is strong on pitch but weak on timbre, and the accuracy drop is concentrated on same-pitch conflict positions. Using only the mel spectrogram (Ch.0 alone) degrades JPA to 93.7%, still far above the 43.2% pitch-only baseline but confirming that the multi-channel design accounts for approximately 60% of the gap between baseline and full accuracy.

Joint classification provides a consistent advantage. The dual-head model trails by 2.6 points, consistent with the theoretical argument that joint classification forces timbral discrimination at the output layer. The auxiliary heads in the full model provide a smaller but measurable 0.5% benefit, suggesting that the additional supervisory signal is helpful but not the primary source of improvement.

Residual connections matter. Removing residual skip connections reduces JPA by 2.2 points, confirming that the skip connections help gradient flow through the deeper layers of the backbone. Without residual connections, the model converges to a lower accuracy plateau at roughly epoch 45–50 rather than continuing to improve.

Augmentation is collectively important. Individually, each augmentation strategy contributes between 0.5 and 1.5 percentage points. The waveform-level augmentations (gain, noise, time-shift) collectively account for the largest single augmentation contribution (1.5%), likely because they directly diversify the distribution of input amplitudes and onset positions that the model encounters during training.

7 Discussion and Critical Reflection

7.1 Assessment Against Original Goals

All five objectives stated in Section 1.2 were achieved. A dataset was recorded, a feature pipeline and model were designed, trained, and evaluated, a working interface was implemented, and performance

was measured quantitatively. The project exceeded its stated accuracy targets substantially.

The most important result is the gap between the pitch-only baseline (43.2%) and the full model (99.4%) on the IS test. This 56.2-point gap directly quantifies the value of timbral feature learning and joint classification for resolving string redundancy. Had the model merely exploited pitch information, it would have matched the baseline. The model's near-perfect in-session performance confirms the hypothesis that timbral features, particularly MFCCs and attack transients, carry sufficient discriminative information to resolve all 138 positions under controlled conditions.

The comparison between the dual-head and joint-head models (Table 6) validates the architectural motivation in Section 4.3. The 2.6-point JPA gap is modest, but it is concentrated in the same-pitch conflict positions, which are the central challenge of the problem. The joint model does not simply perform better overall; it is specifically better at the hard cases the system was designed to solve.

7.2 Comparison with Related Work

Direct numerical comparison with prior work is limited by differences in datasets and evaluation protocols. Wiggins [7] reports approximately 90–95% per-string accuracy on an in-session held-out split; the present system achieves 99.4% under equivalent conditions, attributable to the richer five-channel features, joint classification loss, and targeted same-pitch augmentation. SynthTab [5] operates on polyphonic audio and is not directly comparable. The pitch-only baseline (43.2% JPA) vs. full model (99.4% JPA) gap quantifies the contribution of joint timbral classification; this comparison is not made explicitly in the reviewed literature and represents a novel empirical contribution.

7.3 Assessment of Feature Design

The five-channel feature design was motivated by a literature-grounded hypothesis that timbre and attack are the primary cues for string discrimination. The results support this hypothesis. The error pattern, correct-pitch, wrong-string confusions on adjacent plain strings, is consistent with the model failing precisely where timbral cues are weakest. Plain strings 5 and 6 (B3 and E4) are made from the same material and differ primarily in gauge; their MFCC profiles and attack transients are more similar than those of wound versus plain strings.

An informal channel ablation was conducted by disabling individual input channels during training and observing the change in error rate. Removing the attack-window channel produced the largest increase in same-pitch confusion, followed by removing the MFCC channel. The delta-mel channel had the smallest individual effect but contributed meaningfully in combination with the others. These observations are directional rather than definitive, as they were not evaluated on a separate held-out set.

7.4 Reflection on the Dataset Strategy

One unexplored middle path is fine-tuning a synthetically pre-trained model on a small real-audio corpus. This is a natural next step: pre-training on abundant synthetic data to establish pitch-level representations, then fine-tuning on real data to learn timbral string cues. Domain adaptation techniques such as those in SynthTab [5] or feature alignment methods could be applied to bridge this gap more systematically in future work.

7.5 Limitations

Single guitar and signal chain. All training data was recorded on one specific guitar with one specific pickup and interface. Different string gauges, pickup types (single coil vs. humbucker), acoustic guitars, or varying amplification will produce timbral profiles that differ from the training distribution. In the limit, a model trained on a single guitar is essentially a guitar-specific classifier rather than a general-purpose transcription system.

Monophonic restriction. The system cannot process chords or polyphonic passages. Extending to polyphonic input requires separating overlapping partials in the frequency domain, a substantially harder signal-processing and modelling problem. The current gate-based segmenter would also need significant redesign to handle the onset of multiple simultaneous strings.

Session-to-session variability. The OOS accuracy (75.0%) is acceptable for a prototype but would

not meet the standard expected of a practical transcription tool. The most likely cause is session-specific variation in pluck mechanics: the angle and force of the pick, the contact point along the string's length, and string freshness all influence timbre in ways not captured by the current augmentation strategy.

Rhythm and timing. The current system outputs an unstructured sequence of (string, fret) pairs without any timing information. Generating metrically structured tablature with note durations would require onset detection, tempo estimation, and quantisation, a separate and non-trivial processing stage.

Segmentation and tuning. The gate-based segmenter struggles at fast tempos where inter-note silence is insufficient to trigger closure; a learned onset detector would be more robust. The model assumes standard tuning, alternate tunings shift the pitch-string mapping and require retraining or a tuning-conditioned architecture.

Evaluation scope. The evaluation covers a single guitarist on a single electric guitar. Multi-user evaluation across several players and instruments would be needed before making broader performance claims.

7.6 Computational Requirements and Deployment Feasibility

The trained GuitarNetV5 model has approximately 4.7 M parameters and occupies ≈19 MB on disk as a PyTorch float32 state dictionary. This is compact by modern deep learning standards and well within the memory budget of any contemporary laptop or desktop machine, including those without a discrete GPU.

Inference timing was measured on three representative hardware configurations:

Table 11: Per-note inference latency at various hardware configurations (TTA ×9).

Hardware	Processor	Single-run (ms)	TTA×9 (ms)
Desktop (training machine)	RTX 2060 (GPU)	14	62
Desktop (CPU only)	Intel Core i7-10700	38	215
Laptop (CPU)	Intel Core i5-1135G7	55	410

Even on a mid-range laptop CPU, TTA inference completes in approximately 410 ms per note. Since the gate recorder requires at least 400 ms of post-note silence before closing the gate, TTA inference reliably completes before the next note is ready to be processed, maintaining the real-time illusion of the interface. On GPU hardware, inference is effectively instantaneous from the user's perspective.

For deployment without GPU hardware, TTA can be disabled (single-run mode) to reduce latency to approximately 55 ms, with only a modest accuracy penalty as shown in Table 10. This makes the system suitable for deployment on any Python-capable machine with an audio interface, without requiring GPU hardware or specialised dependencies beyond PyTorch and torchaudio.

The HTTP server architecture adds negligible overhead: the JSON serialisation and file-write for each note takes less than 1 ms, and the browser polling interval of 1 s means that even a small latency budget is easily satisfied. The complete system can be launched from a single terminal command and requires no installation beyond standard Python dependencies, making it straightforward to deploy in an educational setting.

7.7 Future Work

Domain-diverse recording corpus. Expanding the dataset to include multiple guitars, different pickup configurations, and microphone-based acoustic captures would increase domain diversity and improve cross-instrument generalisation. Data augmentation with convolution-based pickup modelling, convolving DI recordings with measured impulse responses of real guitar amplifiers and pickups, could augment the training set cheaply without additional recording time.

Sequence-aware decoding. Each note is currently classified independently. A transformer or CRF layer operating over the sequence of predicted positions could exploit musical context. For

example, guitarists following a melody rarely make large string jumps, and incorporating this constraint as a transition penalty in a Viterbi-decoded CRF would reduce same-string-adjacent confusions in continuous performance contexts.

Chord transcription. Moving to polyphonic input is the primary long-term extension. Chord transcription from audio is itself an active research area [17]; extending the present system to multi-label classification over all 138 positions, or a set-valued prediction using a transformer decoder (analogous to MIDI-to-Tab [6]), could borrow representations from that literature. The main challenge beyond the model itself is training data: polyphonic guitar tablature with aligned audio is scarce, and recording it at the scale required for supervised learning is labour-intensive. SynthTab [5] could provide a starting point for polyphonic pre-training, accepting the domain gap as a practical trade-off.

Rhythm capture. Integrating note duration and onset timing would allow the system to produce metrically structured tablature suitable for tab editors and music notation software. Onset detection for monophonic guitar is a well-studied problem [36]; a lightweight spectral-flux or neural onset detector could be applied as a pre-processing step to timestamp each gate-detected note. Quantisation to a musical grid (aligning detected onsets to the nearest beat subdivision given an estimated tempo) could then be performed using dynamic programming, analogous to the quantisation step in MIDI transcription pipelines.

Alternate tuning support. Drop-D, open-G, DADGAD, and other alternate tunings could be handled by a tuning-conditioned architecture: a learned embedding of the open-string pitch vector concatenated to the shared FC layer output is a parameter-efficient way to supply this context.

Adversarial domain adaptation. A domain discriminator trained to distinguish recording sessions would encourage the backbone to learn session-invariant timbral representations [37], improving out-of-session generalisation without additional labelled data.

User calibration. A short per-guitar calibration procedure (recording one note per string and using these recordings to adapt the model's class representations via nearest-centroid classification or lightweight fine-tuning) could substantially reduce the OOS accuracy gap without requiring full retraining.

8 Conclusion

This project set out to automate the conversion of monophonic guitar audio into tablature notation, addressing the specific challenge of string-fret ambiguity that standard pitch detection cannot resolve. A purpose-built dataset of 11,802 real guitar recordings across all 138 string-fret positions was collected, providing a domain-matched training corpus. A five-channel spectral feature representation combining mel spectrogram, harmonic blend, delta-mel transient, MFCC, and attack-window channels was designed to expose the timbral cues that differentiate strings. GuitarNetV5, a residual convolutional network with joint 138-way position classification, was trained using multi-task loss, same-pitch upsampling, and a broad augmentation strategy.

The system achieves **99.4%** joint position accuracy on the in-session test split, compared to 43.2% for a pitch-only baseline and 96.8% for a dual-head architecture. On the more challenging out-of-session generalisation test, accuracy is 75% overall, ranging from 88% on the wound low-E string to 62% on the plain high-e string. The trained model is integrated into a complete browser-based application providing live gate-based capture, TTA inference, and real-time scrolling tablature display.

Three key insights emerge: real-audio data is preferable to synthetic for pickup-based transcription; joint 138-way position classification outperforms independent string and fret heads for same-pitch disambiguation; and explicit timbral features, particularly MFCC and the attack window, account for the majority of the model's advantage over pitch-only approaches.

Future work should focus on multi-guitar generalisation, sequence-aware decoding, and extending the system towards polyphonic chord transcription. Within its intended scope, the system demonstrates that end-to-end guitar tablature transcription from monophonic audio is practically achievable at high accuracy with a compact model and a carefully designed real-audio dataset.

References

[1] Y. Jadhav, A. Patel, R. H. Jhaveri, and R. Raut, "Transfer learning for audio waveform to guitar chord spectrograms using the convolution neural network," *Mobile Information Systems*, vol. 2022, p. 8544765, 2022.

[2] T. Chesney, "Other people benefit. I benefit from their work. Sharing guitar tabs online," *J. Computer-Mediated Communication*, vol. 10, no. 1, 2004.

[3] R. Macrae and S. Dixon, "Guitar tab mining, analysis and ranking," in *Proc. 12th Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2011, pp. 453–458.

[4] H. Bitteur, "On the use of a guitar tablature editor," in *Proc. Int. Computer Music Conf. (ICMC)*, 2008.

[5] Y. Zang, Y. Zhong, F. Cwitkowitz, and Z. Duan, "SynthTab: Leveraging synthesized data for guitar tablature transcription," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 1286–1290.

[6] D. Edwards, X. Riley, P. Sarmento, and S. Dixon, "MIDI-to-Tab: Guitar tablature inference via masked language modeling," *arXiv preprint arXiv:2408.05024*, 2024.

[7] A. F. Wiggins, "Guitar tablature estimation with a convolutional neural network," in *Proc. Int. Society for Music Information Retrieval (ISMIR)*, 2019.

[8] A. Matos, D. Torres, A. Takahashi, and R. Paradeda, "AutoTab: An interactive system for automatic guitar tablature transcription," in *Proc. 52nd Ann. Seminar on Computer Systems (SEMISH)*, 2025, pp. 275–286.

[9] R. Kehling, J. Abeßer, C. Dittmar, and G. Schuller, "Automatic tablature transcription of electric guitar recordings by estimation of score- and instrument-related parameters," in *Proc. 17th Int. Conf. Digital Audio Effects (DAFx-14)*, Erlangen, Germany, 2014.

[10] F. Morsi and J. A. Morales, "EGIPT: Electric guitar isolated pitch tones dataset," *Data in Brief*, vol. 48, p. 109203, 2023.

[11] P. Sarmento, A. Kumar, C. J. Carr, Z. Zukowski, M. Barthet, and Y.-H. Yang, "DadaGP: A dataset of tokenized GuitarPro songs for sequence models," *arXiv preprint arXiv:2107.14653*, 2021.

[12] E. Benetos, S. Dixon, Z. Duan, and S. Ewert, "Automatic music transcription: An overview," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2019.

[13] M. Schedl, E. Gómez, and J. Urbano, "Music information retrieval: Recent developments and applications," *Foundations and Trends in Information Retrieval*, vol. 8, no. 2–3, pp. 127–261, 2014.

[14] E. J. Humphrey, J. P. Bello, and Y. LeCun, "Moving beyond feature design: Deep architectures and automatic feature learning in music informatics," in *Proc. Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2012, pp. 403–408.

[15] K. W. Cheuk, K. Agres, and D. Herremans, "The impact of audio input representations on neural network based music transcription," *arXiv preprint arXiv:2001.09989*, 2020.

[16] Magenta Team, "Music transcription with transformers," https://magenta.withgoogle.com, 2021.

[17] M. McVicar *et al.*, "Automatic chord estimation from audio: A review of the state of the art," *IEEE/ACM Trans. Audio, Speech, and Language Processing*, vol. 22, no. 2, pp. 556–575, 2014.

[18] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[19] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Trans. Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.

[20] M. Müller, *Fundamentals of Music Processing*. Springer, 2015.

[21] T. Lidy and A. Schindler, "CQT-based convolutional neural networks for audio scene classification," in *Proc. Detection and Classification of Acoustic Scenes and Events Workshop (DCASE)*, 2016.

[22] M. Karjalainen, V. Välimäki, and Z. Jánosy, "Towards high-quality sound synthesis of the guitar and string instruments," in *Proc. Int. Computer Music Conf. (ICMC)*, 1993, pp. 56–63.

[23] N. H. Fletcher and T. D. Rossing, *The Physics of Musical Instruments*, 2nd ed. Springer, 1998.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[25] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Int. Conf. Learning Representations (ICLR)*, 2015.

[26] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[27] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[28] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[29] S. Dieleman and B. Schrauwen, "End-to-end learning for music audio," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 6964–6968.

[30] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Int. Conf. Learning Representations (ICLR)*, 2015.

[31] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Int. Conf. Learning Representations (ICLR)*, 2019.

[32] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Int. Conf. Learning Representations (ICLR)*, 2017.

[33] P. Micikevicius *et al.*, "Mixed precision training," in *Int. Conf. Learning Representations (ICLR)*, 2018.

[34] D. S. Park *et al.*, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019, pp. 2613–2617.

[35] J. Salamon and J. P. Bello, "Deep convolutional neural networks and data augmentation for environmental sound classification," *IEEE Signal Processing Letters*, vol. 24, no. 3, pp. 279–283, 2017.

[36] S. Böck, F. Krebs, and M. Schedl, "Evaluating the online capabilities of onset detection methods," in *Proc. Int. Society for Music Information Retrieval Conf. (ISMIR)*, 2012, pp. 49–54.

[37] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 1180–1189.

[38] Amplified Parts, "Guitar diagram," *Amplified Parts Tech Articles*, [Online]. Available: https://www.amplifiedparts.com/tech-articles/guitar-diagram [Accessed: Apr. 2026].

A Software Module Summary

Table 12: Software components and their roles.

Module	Role
`guitar_trainer_v6.ipynb`	Training pipeline: dataset loading, feature extraction, augmentation, training loop, validation, and checkpoint saving. Produces `best_v5.pt`.
`predict.py`	Inference API: loads checkpoint, extracts five-channel features, applies TTA×9, and returns predicted (string, fret) with confidence and top-3 alternatives.
`guitar_capture.py`	Gate-based live recorder: noise-floor calibration, chunk-by-chunk level monitoring, pre-roll buffering, background save/inference worker.
`main.py`	HTTP application server (localhost:8765): REST endpoints for device enumeration, session start/stop, and session JSON retrieval.
`guitar_tab.html`	Single-page browser interface: home screen with session management and device selection; tab viewer with real-time scrolling tablature grid.

B Glossary

Tablature (tab)
A notation system for guitar encoding string number and fret position rather than abstract pitch.

String redundancy
The property of the guitar whereby many pitches can be played on multiple strings at different fret positions.

Monophonic
Audio in which only one note sounds at any given time.

Mel spectrogram
A time-frequency representation where frequency is mapped to the mel (perceptual) scale.

MFCC Mel-Frequency Cepstral Coefficients: compact spectral-envelope features used for timbre modelling.

SpecAugment
Data augmentation that randomly masks frequency and time bands of the input spectrogram during training.

TTA Test-Time Augmentation: averaging predictions over multiple perturbed versions of an input to reduce variance.

AdamW Adam optimiser with decoupled weight decay regularisation.

SGDR Stochastic Gradient Descent with Warm Restarts; a cosine annealing learning rate schedule with periodic resets.

DI Direct Input; a recording method connecting an instrument directly to an audio interface, bypassing microphones and room acoustics.